SECURITY AUDIT REPORT

Prepared for:

TrancheFi

Prepared by:

Austin Lim

24/3/2026

Project Information

Project Name: TrancheFi
Repository: https://github.com/TrancheFiProject/tranchefi-contracts
Commit Hash: 93bc60fb740ee89f8fb2654af9ab64f4ec6338dd
Audit Date: 24/03/2026
Auditor: Austin Lim
Scope: Smart contracts under /src including Vault and Adapters

---

Severity Classification

Critical — Direct loss of funds or complete protocol failure
High — Major functional or economic vulnerability
Medium — Risk to protocol behavior or user funds under certain conditions
Low — Minor inefficiencies or best practice violations
Informational — Code quality or optimization suggestions

Audit Scope Summary

- Vault logic (deposit, redeem, epoch settlement)
- Adapter integrations (Aave V3, Morpho)
- Tranche mechanics (Senior / Junior)
- Queue processing (ERC-7540 style flows)
- External integrations (Curve, lending protocols)

There were **15** issues found, **5** of which were deemed to be 'high', and **6** of which were rated as 'medium', and **3** of which were rated as 'low', and **1** of which were rated as 'informational'.

Severity Rating	Number of Original Occurrences	Number of Remaining Occurences
CRITICAL	0	0
HIGH	5	0
MEDIUM	6	0
LOW	3	0
INFORMATIONAL	1	0

Project Information ... 1
Severity Classification ... 1
Audit Scope Summary ... 1
1. Circular dependency makes Vault + Adapter deployment impossible ... 3
Description ... 3
Impact ... 3
Recommendation ... 3
2. Identical tranche token names/symbols across vaults cause asset confusion ... 3
Description ... 3
Impact ... 4
Recommendation ... 4
3. Repeated Redundant Zero Checks in Deposit Functions ... 4
Description ... 5
Impact ... 5
Recommendation ... 5
4. requestDeposit() Missing Post-Bootstrap Validation ... 5
Description ... 5
Impact ... 5
Recommendation ... 6
5. Redundant Conditional Logic in Pending Deposit Updates ... 6
Description ... 6
Impact ... 7
Recommendation ... 7
6. Missing Event Emission in cancelDeposit() ... 7
Description ... 7
Impact ... 7
Recommendation ... 7
7. Senior Deposits Allowed Before Junior Seeding During Bootstrap ... 8
Description ... 8
Impact ... 8
Recommendation ... 8
8. Bootstrap Phase Can Be Permanently Locked by Minimal Deposits ... 9
Description ... 9
Impact ... 9
Proof of Concept (Conceptual) ... 10
Recommendation ... 10
9. settleEpoch() May Fail Due to Unbounded Processing (Gas Limit DoS) ... 10
Description ... 11
Impact ... 11
Proof of Concept (Conceptual) ... 11
Recommendation ... 12
10. Possible optimization in _processDeposits() ... 12
Description ... 12

Recommendation..... 13
11. Deposits into a wiped tranche can become permanently stuck..... 13
Description..... 13
Impact..... 13
Recommendation..... 13
12. Withdrawals Larger Than Epoch Cap Can Become Permanently Unfulfillable.. 14
Description..... 14
Impact..... 14
Recommendation..... 14
13. Rescue Function in Adapters Is Unusable From the Vault Flow..... 14
Description..... 15
Impact..... 15
Recommendation..... 15
14. Static Slippage Calculation in Stablecoin Swaps Enables Value Leakage and MEV Exploitation..... 15
Description..... 15
Attack Scenario (MEV / Frontrunning)..... 16
Impact..... 16
Recommendation..... 16
15. Leverage Adjustment Allowed During Oracle Circuit Break..... 16
Description..... 17
Impact..... 17
Attack / Failure Scenario..... 17
Recommendation..... 17

1. Circular dependency makes Vault + Adapter deployment impossible

Severity: High
Affected lines: `TrancheFiVault.sol:L347`, `MorphoAdapter.sol:L82`, `AaveV3Adapter.sol:L58`

Description

The vault constructor requires a lending adapter address, while both adapter constructors require the vault address. In the vault constructor, `_lendingAdapter` is injected during deployment and assigned immediately. In both adapters, `_vault` is also required during construction and stored as an immutable.

This creates a circular dependency that cannot be satisfied in a normal deployment flow.

Impact

Neither side can be deployed first, so the protocol cannot be initialized as designed. This is a hard deployment blocker.

Recommendation

Remove the constructor cycle. The cleanest fix is a two-step setup:

- deploy vault and adapter without cross-linked constructor params
- set counterpart addresses once through a protected initializer or one-time setter

---

2. Identical tranche token names/symbols across vaults cause asset confusion

Severity: Medium
Affected lines: `TrancheFiVault.sol:L362-L363`

Description

Each vault deploys its own Senior and Junior tranche tokens inside the vault constructor, but the token name and symbol are hardcoded and do not include any identifier for the underlying asset or vault instance.

That means a Saturn vault, an Apyx vault, and any future vault would all mint tranche tokens with the same visible metadata even though they represent different assets and risk profiles.

This matters more because the whitepaper explicitly describes a multi-vault architecture with multiple independent vault instances for different underlying instruments, and says future instruments will also be supported by deploying additional vault instances without modifying the core contract.

Impact

This does not usually break core accounting onchain, because the token contract addresses are different, but it creates a real integration and UX risk:

- wallets, block explorers, and portfolio tools may show several unrelated assets with the same name/symbol
- users may mistake one vault's tranche token for another
- offchain accounting, indexing, and exchange/listing integrations become more error-prone
- support and operational mistakes become more likely when multiple vaults are live

In a multi-vault product, identical metadata weakens asset differentiation and can lead to users interacting with the wrong tranche token.

Recommendation

Include a vault-specific or underlying-specific identifier in both token name and symbol at deployment time.

For example:

- `TrancheFi Saturn Senior / tfSATURN-S`
- `TrancheFi Saturn Junior / tfSATURN-J`
- `TrancheFi Apyx Senior / tfAPYX-S`
- `TrancheFi Apyx Junior / tfAPYX-J`

A clean pattern is to pass `vaultName`, `seniorName`, `seniorSymbol`, `juniorName`, and `juniorSymbol` into the constructor so each deployment is unambiguous.

That keeps the multi-vault architecture aligned with the token metadata exposed to users and integrators.

---

3. Repeated Redundant Zero Checks in Deposit Functions

Severity: Low
Affected lines:
`TrancheFiVault.sol:L403-L404`

`TrancheFiVault.sol:L476-L477`
`TrancheFiVault.sol:L497-L498`

Description

Each of these functions performs two validations:

- explicit zero check
- minimum deposit check

Since `MIN_DEPOSIT > 0`, the second condition already enforces non-zero input. Therefore, the explicit zero check is redundant in all cases.

Impact

- no security impact
- minor gas inefficiency due to redundant condition
- slight increase in code complexity and maintenance overhead

Recommendation

Remove the redundant zero check and rely solely on the minimum deposit validation.

---

4. requestDeposit() Missing Post-Bootstrap Validation

Severity: Medium

Description

The whitepaper specifies that `requestDeposit()` is the post-bootstrap deposit path and should only be used once both senior and junior tranches have been funded. However, the contract does not enforce this prerequisite.

As a result, asynchronous deposits may be accepted before bootstrap completion, violating the documented deposit-state transition.

Impact

Violation of protocol lifecycle design:
The contract allows `requestDeposit()` to be called before both tranches are funded, breaking the intended bootstrap → post-bootstrap transition defined in the whitepaper.

Inconsistent accounting during bootstrap:
Async deposits may enter the queue before NAV for both tranches is properly established, which can lead to:

- incorrect share pricing
- unfair allocation between early and later participants

Potential economic imbalance:
Allowing deposits before junior capital is established may weaken the intended subordination structure, especially for senior tranche participants.

Unexpected behavior for integrators and users:
Off-chain systems and users relying on the documented flow may assume `requestDeposit()` is only valid post-bootstrap, leading to integration bugs or incorrect assumptions.

Recommendation

Enforce the bootstrap completion condition explicitly in `requestDeposit()`:

```
require(seniorNAV > 0 && juniorNAV > 0, "bootstrap not complete");
```

---

5. Redundant Conditional Logic in Pending Deposit Updates

Severity: Low
Affected lines:
`TrancheFiVault.sol:L461`
`TrancheFiVault.sol:L463`
`TrancheFiVault.sol:L812`
`TrancheFiVault.sol:L814`

Description

The contract conditionally subtracts assets from pending deposit totals using:

```
pendingX = pendingX > assets ? pendingX - assets : 0;
```

However, by design:

- `assets` corresponds to a processed portion of already recorded pending deposits
- therefore, `pendingSeniorDeposits >= assets`
- and `pendingJuniorDeposits >= assets`

should always hold true.

This makes the conditional check unnecessary, as the subtraction should never underflow in valid execution paths.

Impact

- no direct security impact
- minor gas inefficiency
- redundant logic that complicates readability
- weakens invariant clarity, since it suggests underflow is expected when it should not be

Recommendation

Replace with direct subtraction:

- `pendingSeniorDeposits -= assets;`
- `pendingJuniorDeposits -= assets;`

---

6. Missing Event Emission in cancelDeposit()

Severity: Low

Description

The `cancelDeposit()` function performs state changes related to user deposits, such as updating pending deposits, modifying request state, and potentially transferring assets, but does not emit an event.

In DeFi systems, deposit lifecycle actions typically include events for:

- deposit requested
- deposit processed
- deposit cancelled

While events exist for other actions, such as deposit requests or claims, the absence of an event for cancellation creates an incomplete event trail.

Impact

- reduced transparency: off-chain systems such as indexers, dashboards, and analytics tools cannot reliably track deposit cancellations
- poor user experience: wallets and UIs cannot reflect cancellation activity accurately
- difficult debugging and monitoring: lack of event logs makes it harder to trace user actions and protocol state changes
- inconsistent event design: other lifecycle actions emit events, but cancellation does not

Recommendation

Emit an event when a deposit is cancelled.

---

7. Senior Deposits Allowed Before Junior Seeding During Bootstrap

Severity: Medium
Affected lines:
`TrancheFiVault.sol:L478`

Description

The code comment states:

"Junior should be seeded FIRST to establish subordination before senior deposits."

However, the actual condition:

```
require(seniorNAV == 0 || juniorNAV == 0, ...);
```

allows:

- `seniorNAV == 0 AND juniorNAV == 0` → senior deposit allowed
- `seniorNAV == 0 AND juniorNAV > 0` → allowed
- `seniorNAV > 0 AND juniorNAV == 0` → blocked

This means senior deposits are allowed even when `juniorNAV == 0`, which directly contradicts the intended design described in the comment and whitepaper.

Impact

- violation of core risk model: senior tranche is supposed to be protected by junior capital
- incorrect bootstrap sequencing: allows invalid protocol state
- economic risk to early senior depositors: they may unknowingly take on junior-level risk
- spec deviation: implementation contradicts documented design and comments

Recommendation

Enforce strict junior-first seeding:

```
require(juniorNAV > 0, "junior not seeded");
```

---

8. Bootstrap Phase Can Be Permanently Locked by Minimal Deposits

Severity: High

Description

During bootstrap, deposits are controlled by:

```
require(seniorNAV == 0 || juniorNAV == 0, "use requestDeposit after bootstrap");
```

This means:

- bootstrap phase is active while either tranche has zero NAV
- bootstrap phase ends once both `seniorNAV > 0` and `juniorNAV > 0`

However, there is no restriction on deposit size beyond `MIN_DEPOSIT`.

An attacker can:

- deposit minimum amount into Senior
- deposit minimum amount into Junior

Now:

- `seniorNAV > 0`
- `juniorNAV > 0`
- bootstrap phase ends immediately

Impact

Premature bootstrap termination:
The protocol exits bootstrap before meaningful liquidity is established.

Prevents intended seeding:
Legitimate users cannot deposit larger amounts via `depositSenior()` or `depositJunior()` anymore.

Forces async deposit flow too early:
Users are forced into `requestDeposit()` before proper NAV initialization.

Economic imbalance:
Initial NAV and share pricing may be based on negligible liquidity.

Griefing attack vector:
Any attacker can cheaply disrupt protocol launch with minimal capital.

Proof of Concept (Conceptual)

Attacker:

- `depositSenior(MIN_DEPOSIT)`
- `depositJunior(MIN_DEPOSIT)`

Result:

- `seniorNAV > 0`
- `juniorNAV > 0`
- bootstrap phase ends

Legitimate users:

- cannot use `depositSenior / depositJunior` anymore
- forced into `requestDeposit()`

Recommendation

Enforce stronger bootstrap conditions before transitioning phases.

Option 1: Minimum NAV threshold

```
require(

  seniorNAV >= MIN_BOOTSTRAP && juniorNAV >= MIN_BOOTSTRAP,

  "bootstrap not complete"

);
```

Option 2: Explicit bootstrap flag

bool public bootstrapComplete;

Set only when sufficient liquidity is reached, and do not trigger it with trivial deposits.

Option 3: Controlled initialization
Allow only admin or whitelisted addresses to seed initial liquidity.

---

9. settleEpoch() May Fail Due to Unbounded Processing (Gas Limit DoS)

Severity: High

Affected Lines
From the repository: `TrancheFiVault.sol — settleEpoch()`

Internal loops processing:

- deposit requests
- withdrawal requests

Description

The `settleEpoch()` function processes all pending deposit and withdrawal requests in a single transaction.

Since:

- the function is called once per epoch, weekly
- users can submit an unbounded number of requests during the epoch
- the total number of requests can grow arbitrarily large

the function may:

- exceed the block gas limit
- revert due to out-of-gas
- become uncallable in practice

Impact

Protocol liveness failure:
If `settleEpoch()` cannot execute, the system cannot:

- process deposits
- process withdrawals
- advance epochs

Funds stuck:
User funds may remain locked in pending state indefinitely.

Denial of Service (DoS):
Attackers can deliberately submit many small requests to increase gas cost and block settlement.

Operational risk:
Even without attackers, high usage can naturally cause failure.

Proof of Concept (Conceptual)

1. Many users, or an attacker, submit thousands of deposit and withdrawal requests.
2. Requests accumulate over a week.
3. `settleEpoch()` tries to process all in one transaction.

4. Gas required exceeds the block gas limit.
5. Transaction reverts.
6. Epoch cannot settle and the system becomes stuck.

Recommendation

Refactor settlement to bounded or incremental processing.

Option 1: Batch processing with limits

```
function settleEpoch(uint256 maxIterations) external {

  // process only up to maxIterations requests

}
```

Option 2: Cursor-based iteration

- store `lastProcessedDepositId`
- store `lastProcessedWithdrawId`
- allow multiple transactions to complete settlement

Option 3: Separate processing functions

- `processDeposits(uint256 limit)`
- `processWithdrawals(uint256 limit)`

Option 4: Pull-based claims (preferred)

- record results
- let users claim individually
- avoid heavy loops entirely

---

10. Possible optimization in _processDeposits()

Severity: Informational

Description

The deposit-processing flow can be made more gas efficient by aggregating processed deposit amounts and calling `deployCapital()` once on the total, rather than repeatedly during iteration.

Additionally, `lastProcessedDepositId` may be advanced directly to `nextDepositRequestId` if all requests in the processed range are guaranteed to be

finalized. This reduces loop overhead and avoids repeated rescanning of already handled entries.

Recommendation

Accumulate processed deposit assets into a single total and call `deployCapital()` once after the loop.

Also set:

```
lastProcessedDepositId = nextDepositRequestId
```

only if all earlier requests are guaranteed to be finalized and no skipped request can remain pending.

---

11. Deposits into a wiped tranche can become permanently stuck

Severity: Medium
Affected lines: `TrancheFiVault.sol:L3441-L3443`, `L3421-L3436`, `L3486-L3493`

Description

During `_processDeposits()`, the contract explicitly skips deposits when:

```
nav == 0 && token.totalSupply() > 0
```

to avoid price manipulation.

However, the skipped request is neither processed nor cancelled. It remains pending with its assets still counted, and the claimant cannot call `claimDeposit()` because `req.processed` stays false.

Impact

Users who deposit into a wiped tranche can have funds stuck indefinitely unless some external recovery path is added.

It also keeps `lastProcessedDepositId` pinned behind that request, causing repeated re-iteration over old entries and degrading operational efficiency over time.

Recommendation

Do not silently continue on this condition. Instead:

- automatically cancel and refund the deposit, or
- mark the request as rejected with a claimable refund path

---

12. Withdrawals Larger Than Epoch Cap Can Become Permanently Unfulfillable

Severity: High
Affected lines: `TrancheFiVault.sol:L717`

Description

The withdrawal queue appears to process requests in an all-or-nothing manner against the epoch withdrawal cap.

If a queued withdrawal request is larger than the remaining epoch capacity, the request is skipped until a later epoch. However, if the request amount itself is greater than the maximum withdrawable cap for any single epoch, then it can never be fully satisfied.

As a result, the request can remain at the front of the queue forever and never complete.

This creates a permanent liveness failure for large withdrawal.

Impact

- a user with a withdrawal request larger than the cap may be unable to withdraw indefinitely
- large requests can become permanently stuck in the queue
- if queue processing is FIFO-like, one oversized request may also block later smaller withdrawals
- this creates a denial-of-service condition in the withdrawal pipeline and breaks expected redemption behavior

Recommendation

Support partial fulfillment of queued withdrawals.

---

13. Rescue Function in Adapters Is Unusable From the Vault Flow

Severity: Medium
Affected lines: `MorphoAdapter.sol:L242`, `AaveV3Adapter.sol:L204`

Description

The adapters appear to contain token rescue functionality, but the vault does not call it or expose an operational path for invoking it as part of the protocol flow.

Because the adapters are controlled through the vault architecture, any funds accidentally left in an adapter may become practically unrecoverable if:

- tokens are transferred directly to the adapter by mistake
- reward tokens or dust accumulate in the adapter
- unsupported assets remain stranded after a failed or partial operation

In other words, the rescue mechanism exists at the adapter level, but the system design does not make it reachable through the main protocol control path.

Impact

- tokens accidentally stranded in adapters may be unrecoverable in practice
- reward tokens, dust, or mistakenly transferred assets may remain permanently locked
- operational recovery is weakened, especially during incident response
- this creates avoidable fund-loss risk for non-core assets held by the adapter

Recommendation

Make the rescue path explicitly reachable and governed.

Possible fixes:

- add a vault-admin function that forwards a rescue call to the active adapter
- allow governance or owner to call adapter rescue through a controlled interface
- document which assets are rescueable and which are intentionally not rescueable

---

14. Static Slippage Calculation in Stablecoin Swaps Enables Value Leakage and MEV Exploitation

Severity: Medium
Affected lines: `TrancheFiVault.sol:L907`, `TrancheFiVault.sol:L913`

Description

The swap functions use a fixed slippage threshold, `1%`, based on input amount, rather than computing the minimum output from the pool's current pricing.

This approach assumes:

- both tokens are approximately $1, stablecoins
- exchange rate remains near `1:1`

However, even for stablecoin pools, the actual output depends on:

- pool liquidity and imbalance
- swap fees
- temporary depegs
- on-chain price movement during transaction execution

Because `minOut` is static and predictable, it does not adequately protect against price manipulation or unfavorable execution.

Attack Scenario (MEV / Frontrunning)

1. A user submits a transaction to swap via `_usdcToUnderlying()`.
2. An attacker observes the transaction in the mempool.
3. The attacker front-runs with a trade that shifts the pool price.
4. The victim transaction executes with a degraded exchange rate, but still satisfies `minOut`, using `1%` tolerance.
5. The attacker back-runs to restore price and capture profit.

Impact

- users receive worse execution prices than expected
- MEV bots can extract systematic profit from swaps
- protocol experiences value leakage over time
- larger transactions are more affected

Even in stablecoin pools, this creates a predictable slippage window that can be exploited.

Recommendation

Fix slippage calculation.

- make slippage configurable instead of hardcoded
- use tighter slippage bounds for stablecoin pairs, for example `0.1%–0.3%`
- optionally add price sanity checks such as oracle or TWAP

---

15. Leverage Adjustment Allowed During Oracle Circuit Break

Severity: High
Affected lines: `TrancheFiVault.sol:L1036`

Description

The protocol includes an oracle circuit breaker mechanism intended to prevent operations when price data is unreliable or unavailable.

However, `settleEpoch()` still allows leverage adjustments to be executed even when the oracle circuit is broken.

Leverage adjustments depend on accurate pricing inputs such as:

- asset valuations
- NAV calculations
- coverage ratio
- borrow and repay decisions

If the oracle is invalid or stale, these values become unreliable. Despite this, the system proceeds with leverage rebalancing.

Impact

Incorrect leverage decisions:
The protocol may:

- over-borrow
- under-collateralize positions
- reduce safety buffers

Increased liquidation risk:
Positions may be pushed closer to liquidation due to incorrect health factor assumptions.

Violation of safety design:
Circuit breakers are intended to halt sensitive operations. Bypassing them defeats their purpose.

Attack / Failure Scenario

1. Oracle becomes stale or manipulated, so the circuit breaker is triggered.
2. `settleEpoch()` is called.
3. The protocol proceeds with leverage adjustment using invalid pricing.
4. The system miscalculates coverage and executes incorrect borrow or repay actions.
5. Market normalizes, and the position becomes unsafe, leading to liquidation or loss.

Recommendation

Block leverage adjustment when the oracle circuit breaker is active.